WHISPERFLOW HACKATHON 2026

24-Hour Hackathon — Problem Statements

PROBLEM STATEMENT 1: AMBIENT VOICE OS

Build the Self-Evolving, Zero-UI Intelligence Layer

The Premise

Typing is a bottleneck. GUIs are crutches. The future belongs to ambient voice intelligence — systems that listen, understand, reason, and act across your entire digital environment without you ever touching a keyboard.

Your challenge: Build an application powered by Wispr Flow where **voice is the sole primary interface**. Not a feature. Not a shortcut. The entire operating paradigm. Your app must demonstrate that Wispr Flow's real-time transcription, auto-editing, context awareness, and command mode can power an autonomous system that operates across multiple contexts, devices, and users.

The Core Problem

1. ZERO-UI MANDATE

Your application must be fully operable via Wispr Flow voice input alone. Navigation, execution, debugging, configuration, and multi-step workflows must all be triggered and controlled through natural speech. If a user has to touch their mouse or keyboard for core functionality, you've failed.

2. CONTEXT POLYMORPHISM

Your system must simultaneously maintain awareness of **at least three distinct application contexts** (e.g., IDE, browser/Slack, documentation tool) and dynamically adapt Wispr Flow's output based on which context is active. The same voice command must produce different actions depending on where the user is focused. Example: saying "refactor the auth logic" in an IDE triggers code changes; saying it in Slack generates a message explaining the refactor.

3. MULTI-PARTY VOICE ORCHESTRATION

Your app must handle **real-time multi-user voice streams**. When 2+ users speak simultaneously in a session, your system must:

- Attribute transcription to the correct speaker
- Resolve conflicting voice commands intelligently
- Merge inputs into a collaborative action (e.g., one user dictates code while another dictates documentation, simultaneously)

4. THE AUTONOMOUS AGENT LOOP

Include an agent component that **acts proactively without direct command**. It must:

- Observe transcribed context continuously via Wispr Flow's streaming API

- Detect intent, ambiguity, or task patterns
- Propose or execute actions before the user explicitly asks (e.g., "You've mentioned deploying three times in the last 2 minutes. Should I prepare the staging pipeline?")

5. CROSS-DEVICE CONTINUITY

Demonstrate seamless handoff between **desktop (Mac/Windows) and mobile (iOS/Android)**. A workflow started via voice on desktop must be continuable via voice on mobile, with shared context, personal dictionary, and snippet state synchronized.

6. ADVERSARIAL AUDIO RESILIENCE

Your system must function under **three audio stress conditions**:

- **Whisper Mode**: Accurate transcription when the user speaks barely above a whisper
- **Code-Switching**: Handle real-time switching between two languages mid-sentence (Wispr Flow supports 100+ languages)
- **Overlapping Speech**: Gracefully handle when multiple people speak at once or when background noise competes with the primary speaker

---

Technical Constraints (The Difficulty Multipliers)

Constraint	Why It's Brutal
<500ms Voice-to-Action Latency	From end of speech to system response must be under half a second. Forces ruthless optimization of Wispr Flow's WebSocket streaming and your inference pipeline.
No Hardcoded Commands	You cannot use simple keyword matching. The system must use semantic understanding of transcribed natural language to infer intent.
Privacy-First Architecture	All voice data handling must respect HIPAA/SOC 2 principles. Show how sensitive transcribed data is processed, stored, or discarded securely.
Self-Healing Dictionary	Your app must dynamically expand its vocabulary based on domain-specific jargon extracted from the user's context, mimicking and extending Wispr Flow's personal dictionary feature.

Judging Criteria

Criteria	Weight	Description
Technical Sadism	30%	Handling brutal constraints under live demo pressure
Voice Paradigm Innovation	25%	Interaction model impossible without voice
Context Intelligence	20%	Clever use of Wispr Flow context awareness across apps/devices
Autonomy	15%	Proactive prediction and execution without explicit instruction
Demo Robustness	10%	Survives live testing with accents, whispers, and interruptions

Suggested Domains

- **DevTools**: A voice-native IDE where you code, debug, and deploy by talking to your environment
- **Healthcare**: A HIPAA-compliant clinical documentation system where doctors dictate patient notes while the agent cross-references EHRs and suggests diagnoses
- **Creative**: A real-time collaborative storytelling engine where multiple authors speak characters into existence simultaneously
- **Accessibility**: A system for users with motor disabilities that controls their entire digital life through adaptive voice commands
- **Crisis Response**: A multi-operator command center where overlapping voice reports from field agents are transcribed, attributed, and mapped automatically

---

Deliverables (24 Hours)

1. **Working Prototype** — Must be demoable live. No video substitutions.
2. **Architecture Diagram** — Show how Wispr Flow's API integrates with your agent, context engine, and multi-party audio pipeline.
3. **Latency Benchmark** — Document your voice-to-action timing.
4. **Failure Mode Analysis** — A 1-page writeup of what breaks and why.

PROBLEM STATEMENT 2: VOICE-NATIVE INCIDENT COMMAND

"When the Datacenter is on Fire, Nobody Has Time to Type"

The Premise

Production is down. PagerDuty is screaming. Three engineers are on a bridge call, shouting over each other. The last thing anyone can do is open a laptop and type kubectl commands. In critical infrastructure moments, **voice is the only interface fast enough** — but it must be safer than SSH.

Your challenge: Build a **Voice-Native Incident Response System** powered by Wispr Flow that lets engineers diagnose, debate, and deploy infrastructure fixes entirely through natural speech. The system must be faster than typing, safer than manual CLI access, and smart enough to resolve conflicting human commands under stress.

The Core Problem

1. ZERO-TOUCH INFRASTRUCTURE CONTROL

Every infrastructure action — scaling pods, rolling back deployments, querying logs, draining nodes, rotating secrets — must be executable via Wispr Flow voice input alone. No keyboard. No mouse. No touch.

2. CONFLICT RESOLUTION IN CHAOS

During an incident, 2+ engineers will simultaneously shout commands:

- "Scale the auth service to 10 replicas!"

- "No, roll back to v2.1.4 NOW!"

Your system must:

- Attribute each command to the correct speaker in real-time
- Detect role-based authority (on-call vs. observer)
- Pause and demand **voice-based consensus** when commands conflict
- Require **multi-factor voice authentication** for destructive actions (e.g., database deletion)

3. THE AUTONOMOUS RUNBOOK AGENT

The system must listen to the incident conversation and:

- Auto-suggest runbook steps based on error patterns in logs
- Pre-stage commands before they're explicitly requested ("I see 500s on the API. I've prepared a rollback to the last stable commit. Say 'execute' to confirm.")
- Maintain an immutable, auditable transcript of who said what and what executed when

4. STRESS-CONDITION AUDIO RESILIENCE

Your system must function when:

- Engineers are **shouting** over each other during outages
- Someone is **whispering** sensitive credentials (Wispr Flow whisper mode)
- Accents and **code-switching** occur under pressure (Wispr Flow supports 100+ languages)

5. SAFETY GUARDRAILS

Implement a **"Dead Man's Voice"** protocol:

- Destructive actions require continuous voice confirmation (not just one word)
- If the speaker's voice pattern changes (someone else takes the phone), escalate to MFA
- Auto-rollback if the system detects panic keywords or contradictory instructions within 30 seconds

---

Technical Constraints (The Difficulty Multipliers)

Constraint	Why It's Brutal
<300ms Voice-to-Action	In a crisis, half a second feels like eternity. Optimize Wispr Flow's WebSocket streaming to the metal.
Immutable Audit Ledger	Every voice command, transcription, and execution must be cryptographically logged for post-mortems.
RBAC via Voice Biometrics	The system must distinguish speakers by voice alone and enforce permissions without pre-enrollment.
Context-Aware Command Parsing	Saying "restart it" must resolve to the correct service based on the last 2 minutes of conversation context, not a hardcoded map.
Offline Fallback	If AWS/GCP APIs are down (the incident), local voice control must still function for cached resources.

Judging Criteria

Criteria	Weight	Description
Crisis Realism	30%	Does it feel like a real war room? Did you simulate production pressure?
Safety Engineering	25%	How well do your guardrails prevent catastrophic voice accidents?
Conflict Resolution	20%	How intelligently does it handle overlapping, contradictory commands?
Latency Under Load	15%	Does it stay under 300ms when 3 people are screaming?
Demo Guts	10%	Will you let judges shout fake commands at it live?

Suggested Domains

- **DevOps/SRE**: Kubernetes voice control with autonomous remediation
- **Network Operations**: BGP routing changes via verbal command during DDoS
- **IoT/Industrial**: Factory floor emergency shutdown systems operated by shouting foremen
- **Security Operations**: SOC voice response to active breaches

---

Deliverables (24 Hours)

1. **Live War Room Demo** — Simulate an incident with 2+ speakers. Judges will shout conflicting commands.
2. **Architecture Diagram** — Show Wispr Flow → Voice Biometrics → RBAC → Infra API pipeline.
3. **Latency Report** — Benchmark voice-to-execution under single-speaker and multi-speaker conditions.
4. **Safety Analysis** — Document 3 ways a malicious or confused voice could break production and how you stopped it.

PROBLEM STATEMENT 3: THE POLYGLOT SWARM MIND

"Build a Brain Where 5 People Speaking 5 Languages Create One Thing"

The Premise

Creativity and engineering don't belong to English alone. A designer in Tokyo speaks Japanese. A developer in São Paulo speaks Portuguese. A PM in Berlin speaks German. They need to build a product together — **right now** — and none of them share a language fluently.

Your challenge: Build a **real-time multilingual collaborative creation engine** powered by Wispr Flow where 5+ users speak simultaneously in different languages, and the system translates, merges, resolves conflicts, and executes a shared creative or technical goal without anyone touching a keyboard.

The Core Problem

1. SIMULTANEOUS MULTILINGUAL STREAMING

5 users speak at the same time in 5 different languages (e.g., Japanese, Spanish, Arabic, French, Hindi). Wispr Flow transcribes all 100+ supported languages in real-time. Your system must:

- Attribute each utterance to the correct speaker
- Translate all streams into a unified semantic representation
- Display/respond in each user's native language

2. SEMANTIC CONSENSUS, NOT JUST TRANSLATION

Translation is table stakes. Your system must:

- Detect when two users are saying the **same thing** in different languages ("aumentar el tamaño" and "サイズを大きくする" both mean "increase the size")
- Detect when they are saying **contradictory** things
- Hold a **voice-based vote** when conflicts arise ("Three of you said blue. Two said red. Blue wins. Confirm?")

3. COLLABORATIVE EXECUTION WITHOUT HUMAN COORDINATION

The swarm's voice outputs must directly manipulate a shared artifact:

- **If coding**: One user dictates a function in Mandarin, another dictates a test in German, a third dictates documentation in English — the system assembles a working codebase
- **If designing**: Voice commands adjust a Figma-like canvas in real-time ("Move the button left" in any language moves it for everyone)
- **If writing**: A shared narrative emerges from simultaneous multilingual storytelling

4. CULTURAL CONTEXT AWARENESS

Words don't map 1:1 across cultures. Your system must:

- Detect formality levels (Japanese keigo vs. casual Spanish)
- Adjust tone when translating commands into execution
- Handle right-to-left (Arabic) and left-to-right mixing in the shared output

5. THE SWARM MEMORY

The system must maintain a **persistent, queryable memory** of everything the swarm has ever said, transcribed by Wispr Flow, and make it searchable by concept (not just keyword) across all languages.

Technical Constraints (The Difficulty Multipliers)

Constraint	Why It's Brutal
<400ms Translate-Execute Loop	Transcribe in Japanese → understand semantically → translate to shared logic → execute → respond in Portuguese. All under half a second.
Zero-Latency Shared State	5 users watching the same canvas/codebase update in real-time with no perceptible delay.

Constraint	Why It's Brutal
Speaker-Isolated Transcription	Wispr Flow must transcribe overlapping speech from 5 people in the same room without cross-contamination.
No English Intermediary	You cannot force all logic through English. The semantic layer must be language-agnostic.
Voice-Only Onboarding	A new user joins the swarm and teaches the system their language/preferences entirely by speaking. No forms. No typing.

Judging Criteria

Criteria	Weight	Description
Translation Depth	25%	Did you go beyond literal translation to semantic understanding?
Swarm Intelligence	25%	How well does the system merge 5 chaotic inputs into one coherent output?
Real-Time Performance	20%	Does it feel instantaneous with 5 simultaneous speakers?
Cultural Nuance	15%	Does it handle formality, idioms, and RTL/LTR mixing gracefully?
Demo Chaos	15%	We will put 5 judges on mic speaking different languages. Survive it.

Suggested Domains

- **Global Software Teams**: Distributed coding with zero language barrier
- **Crisis Response UN Style**: Multi-national relief coordination where every second counts
- **Collaborative Worldbuilding**: 5 authors in 5 languages writing the same novel simultaneously
- **Multilingual Education**: A classroom where students and teachers speak natively, but learn together

---

Deliverables (24 Hours)

1. **Live Swarm Demo** — 3+ judges speaking different languages simultaneously. Show the artifact being built.
2. **Semantic Architecture** — Diagram showing how Wispr Flow transcriptions bypass English to reach a shared semantic layer.
3. **Conflict Resolution Log** — Show 3 examples of contradictory voice inputs and how the system resolved them.
4. **Language Stress Test** — Document performance with at least 3 language families (e.g., Indo-European, Japonic, Semitic).

PROBLEM STATEMENT 4: AMBIENT COGNITIVE OFFLOADING

"Your Voice is a Second Brain — Build the Synapse"

The Premise

The most valuable ideas are lost in the gap between thought and action. You mutter "I should really refactor that auth module" while showering. You half-whisper a bug fix approach while walking to coffee. You ramble through a business strategy while driving. None of it gets captured.

Your challenge: Build an **Ambient Cognitive Offloading System** powered by Wispr Flow that passively listens to your stream-of-consciousness speech, extracts latent intent from fragmented thoughts, and automatically structures them into executable tasks, code, documentation, and knowledge — without you ever giving an explicit command.

The Core Problem

1. FRAGMENT-TO-STRUCTURE PIPELINE

Users will speak in **deliberately broken, incomplete ways**:

- "Uhh, so the auth thing... maybe JWT but shorter expiry? Or no, OAuth2... shit, what did Sarah say?"
- "That bug from yesterday... the one with the null pointer... in the cart service... probably the discount code..."

Your system must:

- Transcribe via Wispr Flow even when the user mumbles, pauses, backtracks, or whispers
- Reconstruct **intent** from grammatically broken speech
- Convert fragments into structured output: Jira tickets, GitHub issues, code comments, calendar events, or draft emails

2. THE LATENT INTENT DETECTOR

The system must distinguish between:

- **Actionable thoughts** ("I need to email the team about the delay") → Create draft email
- **Fleeting observations** ("The sky is nice today") → Ignore or tag as journal
- **Half-formed code** ("Maybe a recursive... no, iterative approach with memoization") → Generate code snippet + explanation
- **Conversational asides** ("Wait, was that my phone?") → Filter out

3. CROSS-SESSION MEMORY & RECALL

Wispr Flow works across Mac, Windows, iOS, and Android. Your system must:

- Maintain a **personal knowledge graph** of everything the user has ever vocalized
- Allow voice queries against past thoughts: "What was that idea I had about caching last Tuesday while I was walking?"
- Surface relevant past context when a similar topic arises again

4. PROACTIVE EXECUTION WITHOUT EXPLICIT COMMANDS

The system must act **before the user finishes thinking**:

- If the user mutters about a meeting conflict 3 times in 10 minutes, auto-suggest rescheduling
- If they vocalize a code pattern repeatedly, auto-generate a reusable snippet
- If they mention a person's name + a task, auto-draft the message

5. PRIVACY-FIRST AMBIENT LISTENING

Because this is always-on:

- All processing must happen **locally** or with HIPAA-grade encryption
- The user must be able to whisper "pause memory" and the system stops recording instantly
- Voice data must be ephemeral unless explicitly promoted to long-term memory by the user's speech patterns

---

Technical Constraints (The Difficulty Multipliers)

Constraint	Why It's Brutal
Mumble Transcription	Wispr Flow handles whispers, but can you handle muttered half-words, false starts, and filler sounds?
<1s Fragment-to-Action	From incoherent vocalization to structured Jira ticket in under a second.
Zero Explicit Commands	The system must infer everything. No "Hey Flow, create a ticket". Just think out loud.
Context Disambiguation	When the user says "that thing", the system must know what "that thing" refers to based on 3 days of prior voice history.
Battery & Thermal Constraints	Always-on voice processing on mobile must not drain the battery. Show optimization.

Judging Criteria

Criteria	Weight	Description
Fragment Intelligence	30%	How well does it reconstruct sense from nonsense?
Proactivity	25%	Does it feel like a second brain or a voice assistant?
Memory Depth	20%	Can it meaningfully recall and connect past vocalized thoughts?
Privacy Architecture	15%	Is the always-on nature defensible and secure?
Creep Factor	10%	Is it useful enough to justify the intimacy? (We want "magical," not "stalker")

Suggested Domains

- **Developer Workflow**: Passive capture of debugging monologues into structured issues and code
- **Executive Decision Support**: Board meeting prep from weeks of scattered voiced ideas
- **Creative Writing**: Converting shower thoughts into plot outlines and character sheets
- **Medical Documentation**: Doctors thinking out loud during rounds, auto-structured into patient notes

---

Deliverables (24 Hours)

1. **Live Monologue Demo** — A judge will ramble incoherently for 2 minutes. Show what your system built from it.
2. **Knowledge Graph Visualization** — Show how 5+ "sessions" of fragmented speech connect semantically.
3. **Privacy Whitepaper** — 1 page on how you prevent always-on voice from becoming a surveillance nightmare.
4. **Recall Demo** — Ask the system about a "thought" from earlier in the hackathon using vague voice queries.

UNIVERSAL HACKATHON RULES

Applies to All Problem Statements

Field	Value
Duration	24 Hours
Team Size	2-4
Primary Technology	Wispr Flow API
Demo Requirement	Live demo mandatory — no video substitutions
Pitch Limit	3 minutes maximum

Universal Judging Criteria

Criteria	Weight	Description
Technical Sadism	30%	Handling brutal constraints under live demo pressure
Voice Paradigm Innovation	25%	Interaction model impossible without voice
Context Intelligence	20%	Clever use of Wispr Flow context awareness

Criteria	Weight	Description
Autonomy	15%	Proactive prediction and execution
Demo Robustness	10%	Survives live testing with accents, whispers, interruptions

---

Universal Deliverables

#	Deliverable	Format
1	Working Prototype	Live demo mandatory
2	Architecture Diagram	Shows Wispr Flow integration
3	Latency Benchmark	Documented voice-to-action timing
4	Failure Mode Analysis	1-page writeup of breaking points
5	3-Minute Pitch	No slides longer than 3 minutes

Bottom line for all problems: Don't build a voice app. Build an **intelligence layer** that happens to breathe through Wispr Flow. If it's not difficult enough that half the teams consider quitting at 3 AM, you're not thinking big enough.